



**GENERALITAT
VALENCIANA**

Conselleria d'Educació,
Cultura i Esport



UNIÓ EUROPEA

Fons Social Europeu
L'FSE inverteix en el teu futur

IES SANT VICENT FERRER

MEMÒRIA PROJECTE DE DESARROLLO APLICACIONES WEB



TÍTOL: Autos EIRO

NOM:xxxxxxxxxxxxxxxxx

DNI: xxxxxxxxxxxxxxxxx

NIA: xxxxxxxxxxxxxxxxxxxxx

CURS: 2025 / 2026

CONVOCATÒRIA: Extraordinària

Tutoria del projecte: XXXXXXXXXXXXXXXXXXXX

INDICE

1.	Introducción.....	2
1.1	Objetivo.....	2
1.2	Herramientas y tecnología.....	2
2.	Análisis de requerimientos	5
2.1	Necesidades del cliente.....	5
2.2	Requerimientos funcionales.....	5
2.3	Elección de la solución y justificación.....	6
3.	Implementación.....	7
3.1	Arquitectura.....	7
3.2	Configuración del entorno.....	7
3.3	Backend.....	13
3.4	Frontend.....	26
3.5	Visualización web.....	34
4.	Fases en la realización.....	38
4.1	Planificación inicial.....	38
4.2	Desarrollo del backend	38
4.3	Desarrollo del frontend.....	39
4.4	Pruebas.....	40
5.	Ampliaciones o mejoras.....	41
6.	Conclusiones.....	43
6.1	Lecciones aprendidas.....	43
7.	Bibliografía.....	44

1. Introducción

1.1 Objetivo.

El objetivo de este proyecto es desarrollar una aplicación web sobre un concesionario de vehículos de ocasión o segunda mano. Esta aplicación permitirá al usuario crearse un usuario, registrarse, consultar vehículos, filtrándolos entre una multitud de filtros e incluso añadir en una lista sus favoritos y tenerlos rápidamente accesibles. En esta aplicación vamos a realizar algún filtrado por el que ayudemos al usuario no solo a escoger un vehículo por precio, kilometraje o potencia. Si no que vamos a realizar filtros por lo que el usuario necesita, un vehículo de trabajo, un vehículo de diversión o un utilitario puro.

Para el usuario administrador podrá gestionar todo lo referido a los vehículos, podrá crear/añadir vehículos en su base, editar vehículos, cambiar el estado de algún vehículo marcándolo como reservado, eliminar vehículos de la plataforma cuando este se venda, eliminar usuarios y muchas mas utilidades que se podrían añadir.

Lo que mas me interesa es crear una app lo mas accesible posible para el usuario, que sea muy intuitiva, sin nada en lo que distraerse, que cuando entre solo se centre en ver vehículos, los que sean, filtrados o sin filtrar, pero siempre en visualización de vehículos y sin noticias ni reviwrs ni nada por el estilo que pueda distraer al usuario a lo que venido.

En esta memoria se mostrará no el proceso paso a paso en sí, si no la elección de las tecnologías usadas, soluciones, decisiones en cuanto a la arquitectura etc...

1.2 Herramientas y tecnología.

En esta aplicación he decidido usar las siguientes tecnologías, pues son las que he aprendido en estos últimos tres o cuatro meses realizando las practicas. Lo que mas me ha gustado de estas tecnologías es la capacidad de organización que tienen y lo eficientes que son, reduciendo mucho el código con abreviaciones y que funciona exactamente igual.

Backend:

- **Java 17 + SpringBoot:** Esta tecnología permite desarrollar aplicaciones muy escalables y con una arquitectura muy clara y organizada basada entre capas. SpringBoot simplifica la configuración y la gestión haciéndolo mucho más eficiente.
- **Spring Data JPA:** Esta tecnología facilita el acceso a la base de datos, evitando código repetitivo gracias a sus repositorios que ya incluyen la mayoría de queries mas usadas. Save, delete, update, getByld, getAll, entre otros...
- **PostgreSQL + Docker:** A sido el contenedor de base de datos sobre el que he estado aprendiendo últimamente. No tengo nada en contra sobre MySql, son bastantes parecidas en lo que yo he usado y tocado, seguramente haya más diferencias a las que yo no he llegado todavía.
- **MapStruct:** Automatiza el mapeo entre entidades y dominios. Convierte automáticamente un dominio en una entidad para poder trabajar sobre la bd por ejemplo sin necesidad de añadir código repetitivo, mediante sus repositorios a la hora de compilar realiza el mapeo automáticamente.
- **Lombok:** Simplifica la escritura de código generando automáticamente getters, setters, constructores. Se pone la anotación al principio (@Getter, @Setter). De esta manera evitas escribirlo y el programa lo realiza por ti. Código más limpio y más legible.
- **Flyway:** Permite gestionar la tablas de manera controlada. De esta manera genero las tablas nada mas arrancar por primera vez el proyecto. No solo tablas, si no que también añadimos de esta manera en nuestro caso "X" users y "X" vehicles a la base de datos. De esta manera ya tenemos ejemplos para hacer funcionar el programa de la manera más eficiente posible.
- **Maven:** Facilita la construcción al compilar y el empaquetado del proyecto.
- **Swagger:** Esta tecnología permite verificar el correcto funcionamiento de cada endpoint. Mediante esta url: <http://localhost:8080/swagger-ui/index.html> y teniendo las dependencias necesarias en el proyecto, conseguimos probar cada endpoint. Ver que nos devuelve, que le enviamos, sin necesidad de conectar el front y resolver cualquier problemas antes de empezar en la parte de front.

Frontend:

- **React + TypeScript + Vite:** Es una biblioteca de JavaScript de código abierto, utilizada para construir interfaces de usuario interactivas y dinámicas. Se basa en componentes reutilizables. TypeScript es una forma de agregar definiciones a bases de código JavaScript. Vite agiliza la construcción y recarga la aplicación.
- **DevExtreme:** Es una biblioteca de componentes UI disponibles para utilizar. Ahora tiempo a la hora de crear y diseñar las formas de cada componente. TagBox, Selects, Botoneras, etc...
- **Axios:** Permite realizar llamadas HTTP de forma sencilla y con manejo de errores.
- **Redux y hooks:** Estas dos tecnologías se utilizan para la gestión de estado. Gestiona un estado y facilitar que todos los componentes puedan acceder a los datos que están en ese estado sin necesidad de por ejemplo volver a realizar una llamada a la API... Por ejemplo, si nada mas hacer login, guardamos en el estado "session" el usuario de la persona que se ha logueado, ahí realizaremos proyecto muestra daw una primera llamada a la API para recoger los datos de ese usuario, se guarda en el estado "session" y mas adelante cuando necesitemos cualquier datos en otra pagina donde sea de ese usuario, redux habrá guardado esos datos y los tendremos disponibles, sin necesidad de volver a llamar a la API de nuevo... Cuando cambiemos de usuario el estado se actualizará...

Conclusión: Con esta tecnología se permite tener una organización clara del proyecto, se minimiza el código repetitivo el cual ponen mucho énfasis en las empresas y favorece la futura escalabilidad.

Requisitos del sistema:

- WSL
- Intelli J Idea
- Docker
- PostgreSQL
- Visual Studio Code

2. Análisis de requerimientos

2.1 Necesidades del cliente.

Las necesidades son las siguientes:

- Registros y gestión de usuarios.
- Consulta y modificación de vehículos
- Posibilidad de buscar cualquier vehículo, filtrar o llenar una lista de favoritos.
- Interfaz clara, intuitiva y fácil de utilizar.

Estas necesidades son las mas importantes.

1º. Porque la aplicación se trata principalmente de poder buscar vehículos en una web, que estén a la venta y pueda cubrir la necesidad del usuario.

2º. Meter una amplia gama de filtrados para que así el usuario final no dude entre que vehículo necesita y tenga que ser asesorado por un especialista, que lo pueda buscar el autónomamente.

3º. Mostrar una interfaz lo mas clara y fácil de utilizar posible. Pues en muchas webs la interfaz tiene muchos datos de mas o noticias, elementos que hacen que el usuario no pueda estar centrado en lo que necesite o le sea difícil de usar.

2.2 Requerimientos funcionales.

- **Registro de usuarios:** El sistema deberá permitir que un usuario se registre con sus datos (nombre, DNI, email, teléfono...).
- **Autenticación de usuarios:** Este debe permitir autenticarse a cada usuario, en este caso mediante un username y un password, para acceder a las funcionalidades del sistema.
- **Gestión de sesión:** El sistema debe mantener proyecto muestra daw la sesión del usuario logueado.
- **Creación de vehículos:** El propietario del concesionario deberá poder registrar vehículos a la aplicación introduciendo datos (marca, modelo, matricula, precio, km...)
- **Eliminación de vehículos:** El propietario una vez se a vendido el vehículo debe poder eliminarlo de la base de datos.
- **Gestión de favoritos:** El usuario final deberá poder tener una lista de sus coches favoritos, para la hora de realizar comparaciones, comprar, preguntar por alguno pueda acceder rápidamente a estos.

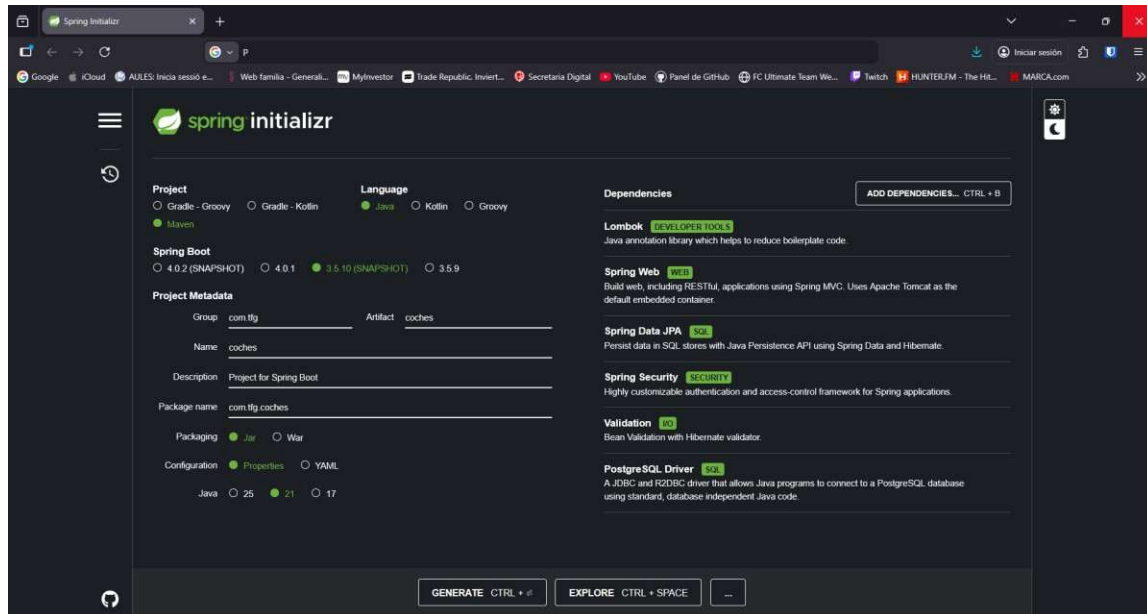
- **Búsqueda y filtrado:** El sistema tiene que permitir el filtrado de vehículos por todos los datos que tiene cada vehículo.
- **Comunicación cliente-servidor:** La aplicación tendrá que intercambiar información entre el frontend y el backend.
- **Validación y gestión de errores:** El sistema debe validar los datos introducidos y mostrar mensajes de error o alertas cuando estos son erróneos.

2.3 Elección de la solución y justificación.

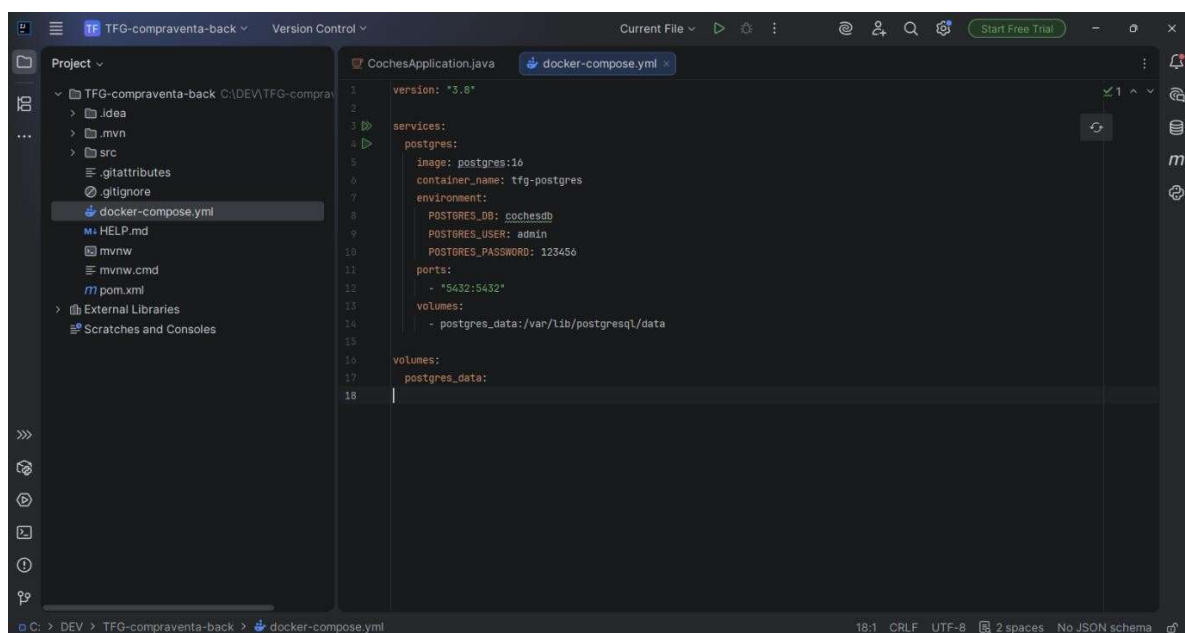
- **Elección tipo de aplicación:** Elegí este tipo de aplicación, una compraventa de vehículos porque es algo con lo que estoy muy metido. Me encantan los vehículos, soy mecánico, observo muchas plataformas de vehículos y cada vez que estoy en alguna siempre veo posibles cosas a mejorar y cosa que se me ocurren que podrían llevar. Es un caso realista y muy común en la sociedad, así que es fácil de entender.
Permite implementar un CRUD bastante completo, pero a la vez complejo ya que se trabaja con un backend completo, frontend, base de datos y queries.
- **Elección de tecnologías:** En cuanto a las tecnologías, debido a mi poca experiencia he escogido estas entre otras porque es lo más reciente que he aprendido. Pero a parte de eso porque me gusta mucho la estructuración y organización que tiene SpringBoot en este caso y JPA para la conexión e integración con la base de datos.
En cuanto a Redux también me han gustado mucho las ventajas que tiene, ya comentadas anteriormente, permite desarrollar y mantener estados sin hacer múltiples llamadas a la API.

PFG DAW Pla Antic Exemple. Títol: Autos EIRO

- Spring Data JPA: Para acceder a la base de datos.
- Spring Security: Para el login y los roles.
- Lombok: Nos ahorramos realizar código de getters y setters mediante @Setter o @Getter al comienzo.
- PostgreSQL: Para conectar la base de datos con el programa.



Lo siguiente que hacemos es abrir el proyecto en el ide que vamos a utilizar, en este caso para el back IntelliJ Idea.



Creemos el documento docker-compose.yml.

Este, es un archivo de configuración del Docker, lo vamos a utilizar para levantar los contenedores, en este caso como se ve en la imagen el de PostgreSQL para la base de datos.

Vamos arrancar PostgreSQL y comprobar que lo estamos haciendo correctamente.

En wsl mediante el comando: docker compose up -d

```
ameneiro9@PC-de-Ameneiro:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
ameneiro9@PC-de-Ameneiro:~$ cd /mnt/c/DEV/tfg-compraventa-back
ameneiro9@PC-de-Ameneiro:/mnt/c/DEV/tfg-compraventa-back$ docker compose up -d
WARN[0000] /mnt/c/DEV/tfg-compraventa-back/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored,
please remove it to avoid potential confusion
[+] up 20/20
✔ Image postgres:16                               Pulled                               43.3s
✔ Network tfg-compraventa-back_default             Created                             0.1s
✔ Volume tfg-compraventa-back_postgres_data        Created                             0.0s
✔ Container tfg-postgres                           Created                             0.8s
ameneiro9@PC-de-Ameneiro:/mnt/c/DEV/tfg-compraventa-back$
```

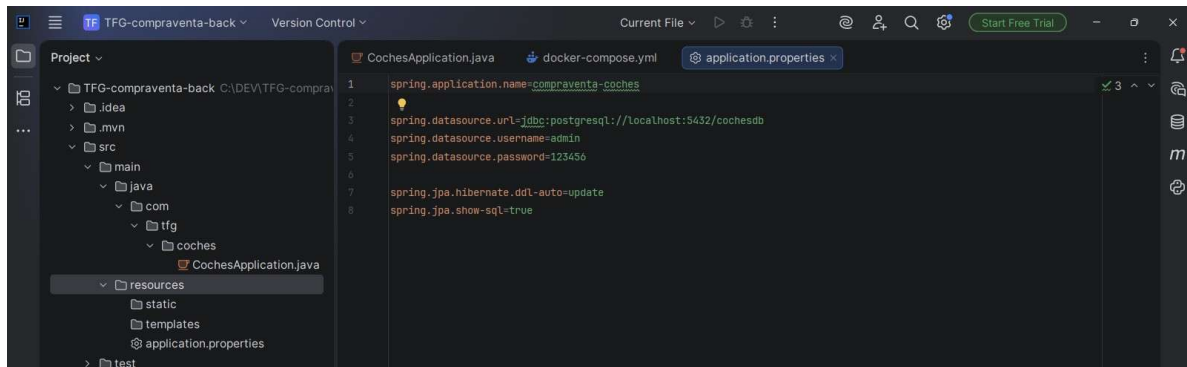
Buscamos el archivo application.properties dentro de la carpeta src/resources.

Este archivo lo que haces es crear una configuración para que la aplicación arranque correctamente.

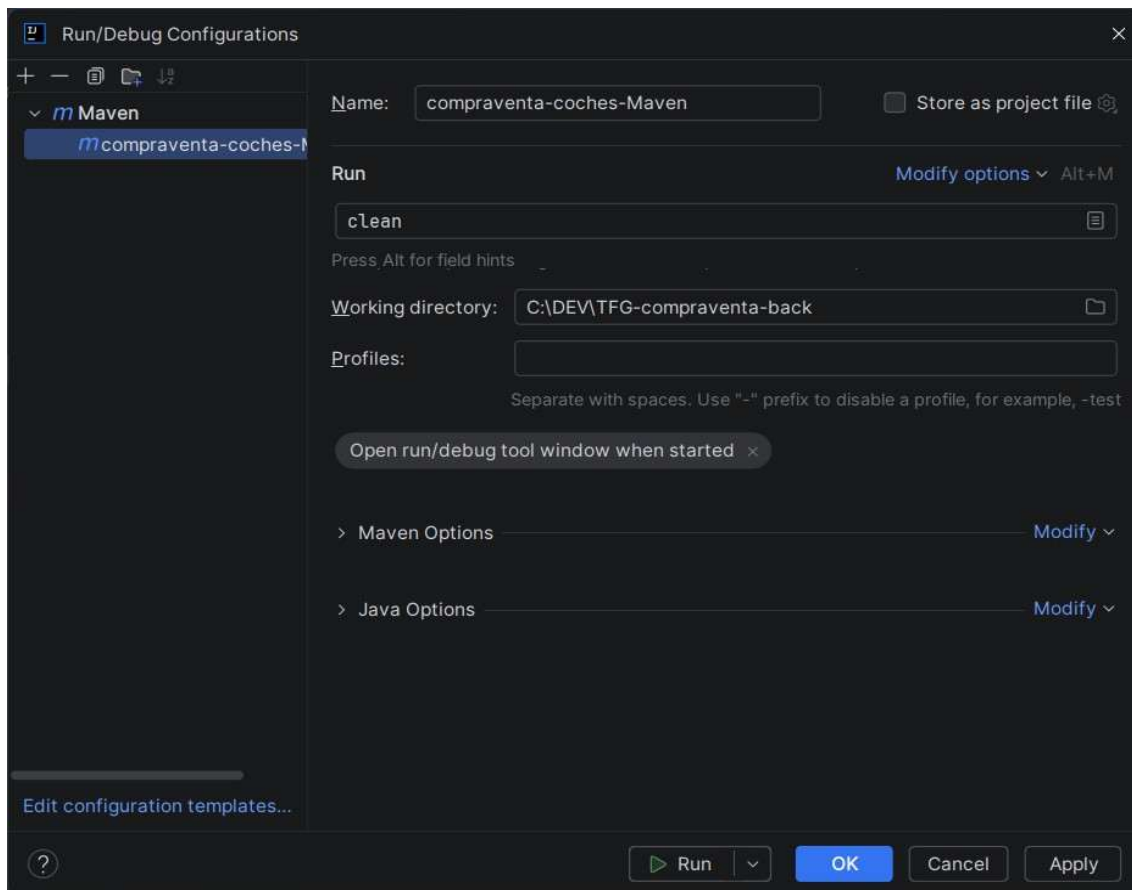
Que base de datos va utilizar.

Usuario y contraseña.

Manejo de las tablas.

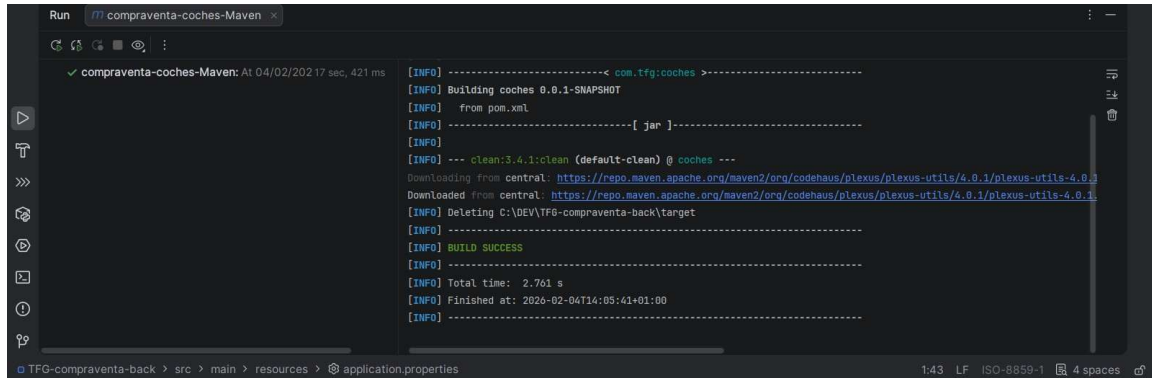


Preparamos la compilación. Separamos la compilación (Maven) del arranque de la aplicación. De esta manera controlamos nosotros la limpieza y compilación de dependencias por un lado y por el otro lado el arranque del servidor...



En el primer “run” con Maven limpia las compilaciones anteriores y compila de nuevo el proyecto y todas las dependencias que hayamos añadido para que estén correctamente descargadas y configuradas.

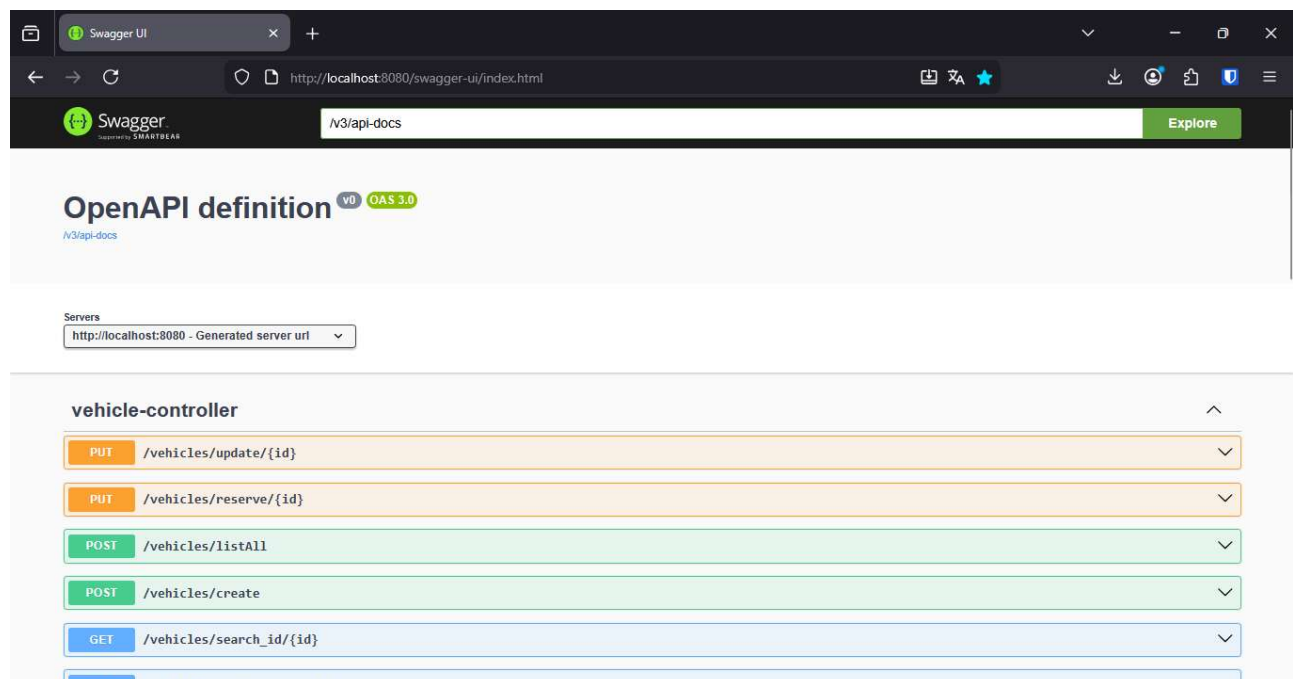
Cada vez que se realice un cambio en el código o se añada algo hay que volver a compilar. Por ejemplo, si añado algún atributo de un objeto y vuelvo a compilar, Maven no creará los setters y getters por lo que no podre utilizarlos y dará error.

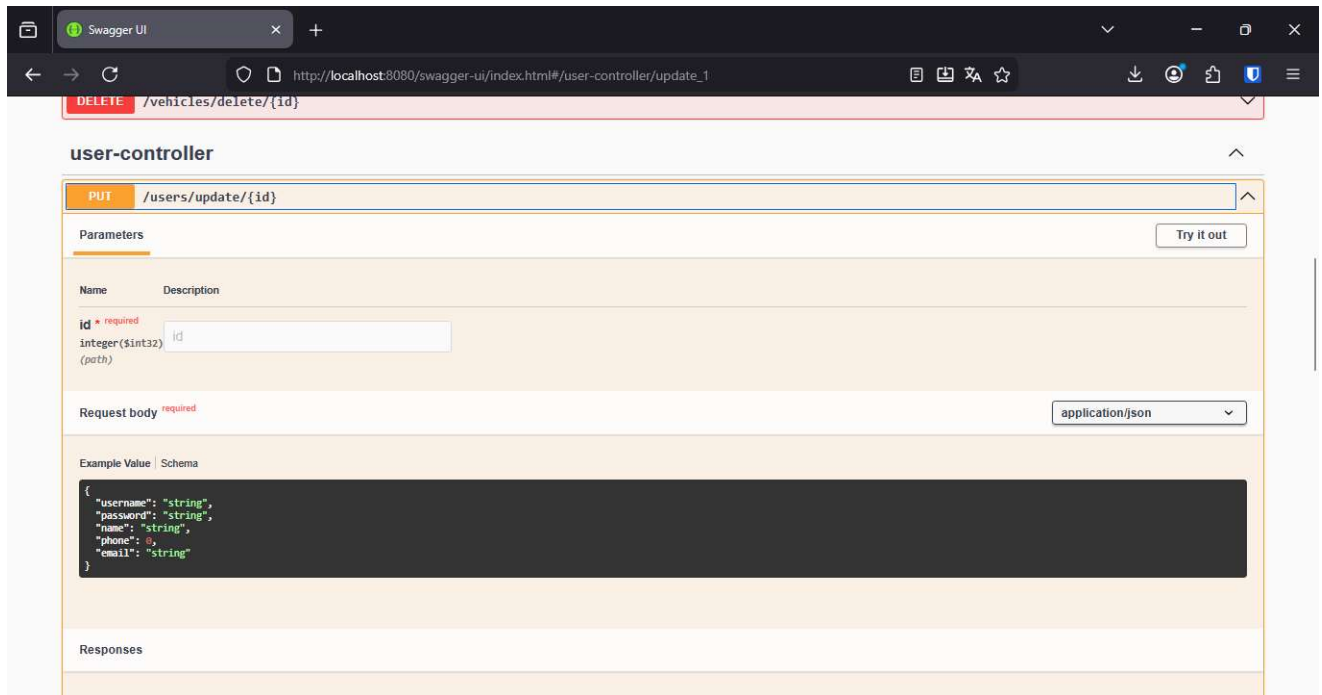


```
Run compraventa-coches-Maven
✓ compraventa-coches-Maven: At 04/02/2021 17:42:17 ms
[INFO] ----- com.tfg.coches -----
[INFO] Building coches 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- clean:3.4.1:clean (default-clean) @ coches ---
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/4.0.1/plexus-utils-4.0.1.jar
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/4.0.1/plexus-utils-4.0.1.jar
[INFO] Deleting C:\DEV\TFG-compraventa-back\target
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.761 s
[INFO] Finished at: 2026-02-04T14:05:41+01:00
[INFO] -----
TFG-compraventa-back > src > main > resources > application.properties 1:43 LF ISO-8859-1 4 spaces
```

Este es el mensaje cuando ha compilado correctamente.

Una vez está el código completo lo proamos con Swagger. Ponemos las dependencias en nuestro POM.xml y las descargamos. Ponemos en el navegador en este caso la siguiente ruta apuntando al puerto que hayamos configurado. <http://localhost:8080/swagger-ui/index.html>

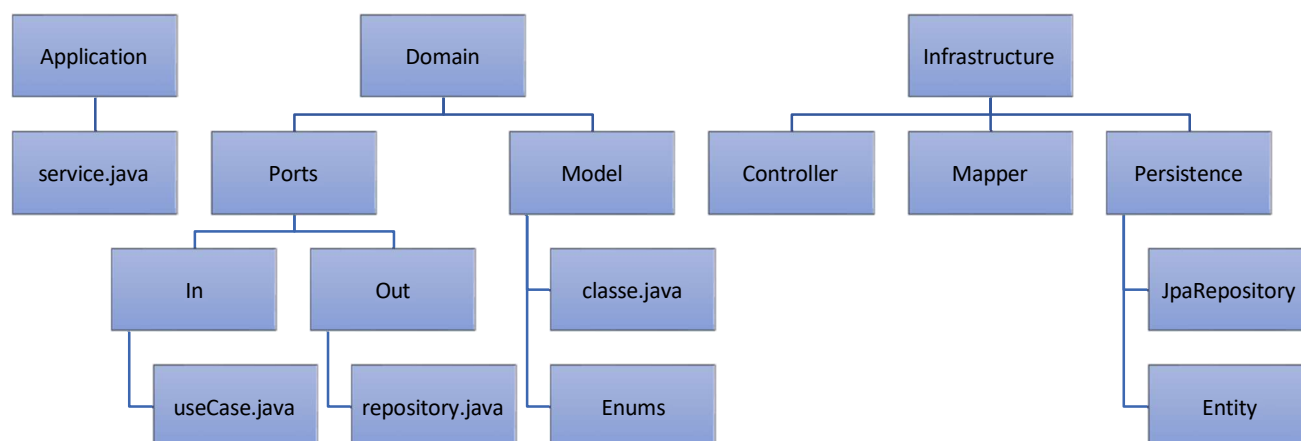




Aquí podemos ver dos imágenes de como se mostraría Swagger y cómo podríamos probar algún endpoint, en este caso el método PUT para actualizar un usuario. Se puede ver como el endpoint espera un "id" y un "request" el cual son los datos a actualizar.

3.3 Backend.

Esquema



Por regla general este va ser el esquema con el que vamos a realizar cada uno de los slices. En el se puede observar que cada slice tiene tres paquetes. Dependiendo del slice puede llevar alguna carpeta más, pues si queremos realizar validaciones por ejemplo, que mas adelante veremos, pues crearemos otra carpeta llamada helper donde incluiremos ese archivo. Si queremos realizar algún update de solo unos datos específicos y no del objeto entero, crearemos un archivo request que ira en una carpeta llamada request, que veremos más adelante... Pero la idea principal y que si o si debe llevar es este esquema.

- **Application:** Este paquete contiene la lógica de la aplicación y los casos de uso. Coordina las operaciones necesarias para cumplir una acción. Es el intermediaria entre el dominio y la infraestructura. Este paquete solo define que pasos hay que seguir para ejecutar un caso de uso.
- **Domain:** Este paquete contiene el núcleo del negocio de la aplicación. En este paquete es donde están los modelos (las entidades principales), los ENUM (que definen valores controlados) y los puertos.

- **Infraestructure:** Este paquete contiene los detalles técnicos. Es la encargada de exponer la aplicación al exterior mediante los “Controllers REST”. Contiene el acceso a la base de datos. Realiza el mapeo entre entidades.

Esta organización por paquetes permite:

1. Separar las responsabilidades.
2. Facilitar el mantenimiento entre capas.
3. Seguir buenas prácticas alineadas con arquitectura hexagonal.
4. Cada slices es independiente y sigue la misma arquitectura.

Entidades y lógica de dominio: Contiene las entidades principales, define las reglas, se crean las interfaces que mas adelante se van a usar por lo tanto define las reglas de la aplicación. Esta contiene los casos de uso.

- **User:** Representa a un usuario registrado. El usuario puede crearse un usuario, borrárselo, editarlo, tener una lista de favoritos y realizar un login.

```
@Getter @lvan
@Setter
public class User {
    private Integer id;
    private String username;
    private String password;
    private String name;
    private String dni;
    private Integer phone;
    private String email;
    private String role;
    private Set<Vehicle> favouriteVehicles = new HashSet<>();
}
```

- **Vehicle:** Representa un vehículo en venta. El objeto vehículo puede crearse y añadirse a la lista de vehículos, puede estar asociado a la lista de favoritos usuarios, se pueden eliminar, se pueden editar. Contiene cuatro enums. Type, Fuel, Transmission, Function.

```
@Getter @Ivan
@Setter
public class Vehicle {
    private Integer id;
    private String brand;
    private String model;
    private Integer power;
    private Integer engineCapacity;
    private String licensePlate;
    private Double price;
    private Integer km;
    private Date dateCreated;
    private String color;
    private String description;
    private Integer doors;
    private Boolean reserved;
    private TypeCar type;
    private FuelCar fuel;
    private TransmissionCar transmission;
    private FunctionCar function;
}
```

```
@Getter 7 usages @Ivan
@AllArgsConstructor
@NoArgsConstructor
public enum TypeCar {
    SEDAN( nameTypeCar: "Sedan"), no usages
    HATCHBACK( nameTypeCar: "Hatchback"), no usages
    SUV( nameTypeCar: "Suv"), no usages
    ALL_TERRAIN( nameTypeCar: "4x4"), no usages
    COUPE( nameTypeCar: "Coupe"), no usages
    VAN( nameTypeCar: "Van"); no usages

    private String nameTypeCar;
}
```

```
@Getter 5 usages @Ivan
public enum FunctionCar {
    WORK, no usages
    UTILITY, no usages
    RACING, no usages
    LUXURY no usages
}
```

```
@Getter 5 usages @Ivan
public enum FuelCar {
    GASOLINE, no usages
    DIESEL, no usages
    ELECTRIC, no usages
    HYBRID no usages
}
```

```
@Getter 5 usages @Ivan
public enum TransmissionCar {
    MANUAL, no usages
    AUTOMATIC no usages
}
```

Application: Actúa como intermediario entre el dominio y las infraestructura. Implementa las funciones designadas por los caso de uso en el dominio.

```
✓ @Service  @ Ivan
  @AllArgsConstructor
  public class VehicleService implements VehicleUseCase {

      private final VehicleRepository vehicleRepository;

      @Override  @ Ivan
      > public Vehicle create(Vehicle vehicle) { return vehicleRepository.save(vehicle); }
```

Como se puede observar se implementa el UseCase (port In), se reciben los datos y se llama al Repository (port Out). Que se implementa en otra clase. Digamos que es el puente como hemos comentado antes entre el domain (ports) y la infrastructure.

Modelo y base de datos: Se utiliza JPA para el mapeo objeto entity. Permite convertir las clases de Java en datos en la base de datos.

@Entity -> Decimos a JPA que esto es una tabla.

@Id -> Indicamos que ese atributo es la clave primaria.

@GeneratedValue -> El Id se generará automáticamente, por eso en el front no lo envió en el cuerpo de la petición, pues se genera automáticamente y auto incremental.

@ManyToOne, OneToMany, ManyToMany. De esta manera se definen las relaciones.

En la siguiente pagina se muestra una imagen de un ejemplo para que se observe como se utilizan las nomenclaturas y como se realizaría un entity, en este caso UserEntity.java

```
@Table(name = "users")
public class UserEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(name = "username")
    private String username;

    @Column(name = "password")
    private String password;

    @Column(name = "name")
    private String name;

    @Column(name = "dni")
    private String dni;

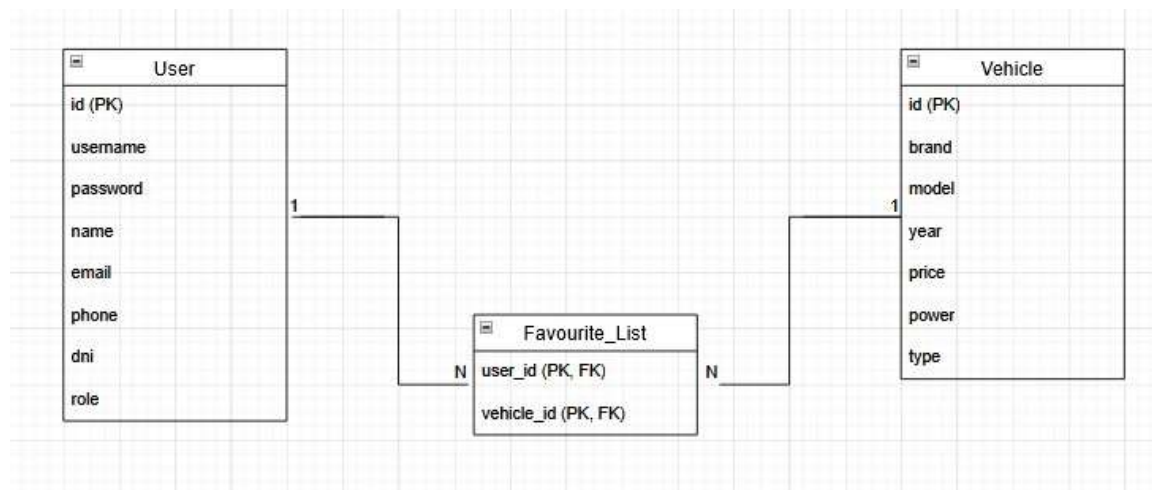
    @Column(name = "phone")
    private Integer phone;

    @Column(name = "email")
    private String email;

    @Column(name = "role")
    private String role;

    @ManyToMany
    @JoinTable(
        name = "favourite_vehicles",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "vehicle_id")
    )
    private Set<VehicleEntity> favouriteVehicles = new HashSet<>();
}
```

Esquema de tabla relacional



Migraciones (Flyway)

Creamos las tablas manualmente desde flyway, de esta manera una vez arranquemos la aplicación se nos crea las tablas que hayamos detallado. Realizaríamos las queries pertinentes, para crear types para los enums, para crear las tablas y para insertar datos y tener ya algo sobre lo que trabajar.

Dentro de /src/main/resources/db.migration creamos 3 archivos, el primero, ha de ser la creación de los tipos y las tablas, sin esta no podemos realizar los siguientes que son los inserts de datos. Pues si no existe tabla no se puede insertar.

V1__create_tables.sql

```

CREATE TYPE TypeCar as ENUM ('SEDAN', 'HATCHBACK', 'SUV', 'ALL_TERRAIN', 'COUPE', 'VAN');
CREATE TYPE FuelCar as ENUM ('GASOLINE', 'DIESEL', 'ELECTRIC', 'HYBRID');
CREATE TYPE TransmissionCar as ENUM ('MANUAL', 'AUTOMATIC');
CREATE TYPE FunctionCar as ENUM ('WORK', 'UTILITY', 'RACING', 'LUXURY');
    
```

```

/*Creacion de tabla favoritos.*/
CREATE table users
(
    id          INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    username    VARCHAR(50) NOT NULL,
    password    VARCHAR(50) NOT NULL,
    name        VARCHAR(50) NOT NULL,
    dni         VARCHAR(9)  NOT NULL,
    phone       INTEGER     NOT NULL,
    email       VARCHAR(50) NOT NULL,
    role        VARCHAR(20) NOT NULL
)

/*Creacion de tabla favoritos.*/
CREATE TABLE favourite_vehicles
(
    user_id     INTEGER NOT NULL REFERENCES users (id),
    vehicle_id  INTEGER NOT NULL REFERENCES vehicles (id),
    PRIMARY KEY (user_id, vehicle_id)
);
    
```

Estos dos ejemplos son creación de los types para los enum y la creación de la tabla Users junto la de favourite vehicles.

V2__insert_users.sql

```
INSERT INTO users (dni, email, name, password, phone, role, username)
VALUES ( dni '12345678A', email 'ameneiro95@email.com', name 'Ivan Ameneiro',
        ( dni '23456789B', email 'arantxafg22@email.com', name 'Arantxa Fonseca',
        ( dni '34567890C', email 'carlos@email.com', name 'Carlos García', pas
        ( dni '45678901D', email 'laura@email.com', name 'Laura Martínez', pas
        ( dni '56789012E', email 'david@email.com', name 'David Sánchez', pass
        ( dni '67890123F', email 'ana@email.com', name 'Ana Rodríguez', passwo
        ( dni '00000000Z', email 'admin@email.com', name 'Administrador', pass
```

V3__insert_vehicles.sql

```
INSERT INTO vehicles
(brand, color, date_created, description, doors, engine_capacity, fuel, function, km, license_plate, model, power,
 price, reserved, transmission, type)
VALUES ( brand 'BMW', color 'Black', date_created '1998-05-12', description 'Clásico deportivo alemán bien conservado',
        license_plate '4832FJK', model 'M3 E36', power 286, price 18000, reserved false, transmission 'MANUAL', type 'C
        ( brand 'Audi', color 'Silver', date_created '2001-09-23', description 'Compacto fiable para uso diario', doors 4,
        license_plate '72916TR', model 'A3', power 101, price 9500, reserved false, transmission 'MANUAL', type 'HATCHB
        ( brand 'Mercedes', color 'Gray', date_created '2003-03-17', description 'SUV amplio y cómodo', doors 5, engine_c
        model 'ML 270', power 163, price 8700, reserved false, transmission 'AUTOMATIC', type 'SUV'),
```

Mapeo

Para conectar la clase java con la base de datos es necesario convertir a entity o a clase java como hemos comentado anteriormente. Aquí entra en juego el mapeo. Se realiza una clase por la que la información pasa y se mapea a entidad o a dominio, si viene de la base de datos hacia fuera habrá que convertirlo para que java lo pueda tratar y viceversa. Pongo un ejemplo, se declara la función y después al compilar el Maven, el se encarga de hacer los getters y setters y la transformación internamente.

```
@Mapper(componentModel = "spring") 3 usages 1 implementation 2 Ivan
public interface VehicleMapper {

    Vehicle toDomain(VehicleEntity entity); 6 usages 1 implementatio

    VehicleEntity toEntity(Vehicle vehicle); 1 usage 1 implementatio

    List<Vehicle> toDomainList(List<VehicleEntity> vehicles); 1

}
```

Repositorios y persistencia

Los repositorios en SpringBoot son interfaces que se encargan de acceder a la base de datos. Se extiende JpaRepository. Gracias a esta extensión SpringBoot genera automáticamente las operaciones básicas, que ya hemos comentado al principio, de un CRUD. (CREATE, READ, UPDATE, DELETE).

De esta manera ya no es necesario que tengamos que realizar más código para hacer inserts, deletes, traer datos básicos... todo ello sin necesidad de escribir SQL manualmente. Ejemplo:

```
findById(Integer id)
```

```
findByEmail(String email)
```

Esto ayuda, pero no es suficiente, cada aplicación requiere de alguna query más que no está en la biblioteca JPA. Entonces ahí ya hay que crearla manualmente.

En nuestro caso, hemos tenido que crear varias queries manuales para el listar las marcas que había en nuestra base de datos por ejemplo, listar los modelos de solo los vehículos que existe en nuestra base de datos y realizar un filtrado para buscar un vehículo por múltiples opciones. De esta manera lo que conseguimos es no hardcodear luego en el back marcas que alomejor no tenemos en nuestra bd, mostramos únicamente las que tenemos de manera automática y si se introduce un nuevo vehículo esa marca aparecerá.

```
@Repository 4 usages 1 Ivan
public interface VehicleJpaRepository extends JpaRepository<VehicleEntity, Integer>, JpaSpecificationExecutor<VehicleEntity> {

    Optional<VehicleEntity> findByLicensePlate(String licensePlate); 2 usages 1 Ivan

    @Query("SELECT DISTINCT v.brand FROM VehicleEntity v") 1 usage 1 Ivan
    List<String> findAllBrands();

    @Query("SELECT DISTINCT v.model FROM VehicleEntity v WHERE v.brand IN :brands") 1 usage 1 Ivan
    List<String> findAllModelsByBrands(@Param("brands") List<String> brands);
}
```

Aquí podemos ver la query manual para buscar marcas de vehículos desde mi base de datos y luego para buscar modelos, que ya existan dependiendo de las marcas de vehículos que tenga.

Mediante SpecificationByFilter de JPA, creamos una query dependiendo solo de los datos que introduzcamos.

Un ejemplo:

```
public static Specification<VehicleEntity> byFilter(VehicleFilter filter) { 1 usage 2 Ivan
    return ( Root<VehicleEntity> root, CriteriaQuery<?> query, CriteriaBuilder criteriaBuilder) -> {
        List<Predicate> predicates = new ArrayList<>();

        if (filter.getBrands() != null && !filter.getBrands().isEmpty()) {
            predicates.add(root.get("brand").in(filter.getBrands()));
        }

        if (filter.getModels() != null && !filter.getModels().isEmpty()) {
            predicates.add(root.get("model").in(filter.getModels()));
        }

        if (filter.getPowerMin() != null) {
            predicates.add(criteriaBuilder.greaterThanOrEqualTo(root.get("power"), filter.getPowerMin()));
        }

        if (filter.getPowerMax() != null) {
            predicates.add(criteriaBuilder.lessThanOrEqualTo(root.get("power"), filter.getPowerMax()));
        }
    }
}
```

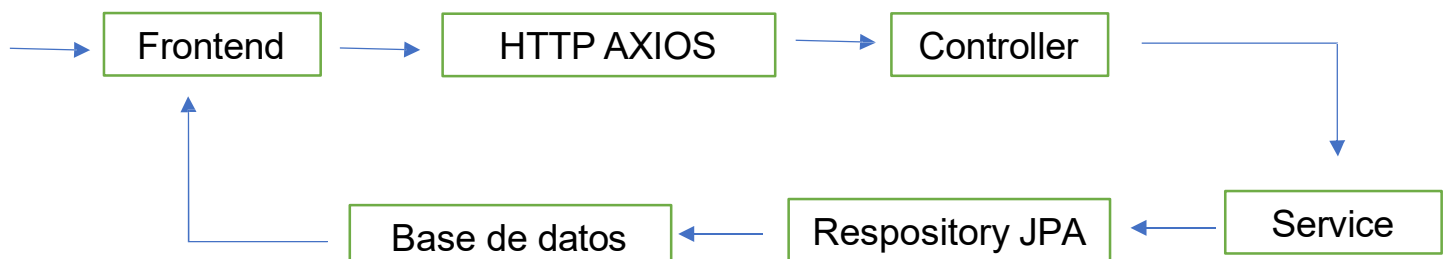
API REST y controller

@RestController digamos que es la puerta de entrada a la aplicación, recibe los datos enviados desde el front mediante las petición HTTP.

Se encarga de:

1. Recibir una solicitud (request, id, param...)
2. Transformar los datos a objetos, llamando a la función mapper antes mencionada.
3. Llama al service (Application) para realizar el método con los datos recibidos.
4. Devuelve la respuesta al usuario.

Flujo:



Los controladores usan anotaciones para mapear las peticiones a métodos. Estas anotaciones son las siguientes y se ponen antes de cada endpoint, junto a el la ruta a seguir.

Anotación	Método HTTP	Uso
@PostMapping	POST	Crear
@PutMapping	PUT	Modificar
@GetMapping	GET	Obtener
@DeleteMapping	DELETE	Eliminar

➤ **POST:** Sirve para crear nuevos recursos.

```
➤ //Añadir vehiculos a la bd por el admin.
@PostMapping("/create")
public ResponseEntity<Vehicle> create(@RequestBody Vehicle vehicle) {
    return ResponseEntity.ok(vehicleService.create(vehicle));
}
```

➤ **PUT:** Sirve para modificar recursos ya existentes.

```
➤ //Actualizar algun vehiculo.
@PutMapping("/update/{id}")
public ResponseEntity<Vehicle> update(
    @PathVariable Integer id,
    @RequestBody VehicleUpdateRequest request) {
    return ResponseEntity.ok(vehicleService.edit(id, request));
}
```

➤ **GET:** Sirve para recuperar información, datos, etc...

```
➤ //Listar todas las marcas disponibles
@GetMapping("/brands")
public ResponseEntity<List<String>> findAllBrands() {
    return ResponseEntity.ok(vehicleService.findAllBrands());
}
```

➤ **DELETE:** Se utiliza para eliminar recursos.

```
➤ @DeleteMapping("/delete/{id}")
public ResponseEntity<Vehicle> delete(@PathVariable Integer id) {
    vehicleService.delete(id);
    return ResponseEntity.noContent().build();
}
```

Eso son los básicos, por ejemplo si queremos realizar una llamada para que el estado de un vehículo reservado, que lo hemos indicado como booleano, pase a estar true o de vuelta a false, haríamos algo así:

```
@PutMapping("/reserve/{id}")  Ivan
public ResponseEntity<Vehicle> reserve(@PathVariable Integer id) {
    return ResponseEntity.ok(vehicleService.reserve(id));
}
```

Puesto que es una modificación de un recurso usamos el método PUT, pasándole una variable que es el id. En este caso llamaría al Service, llamaría a esa función, y le pasaría a la base de datos ese Id, la base de datos recuperaría ese vehículo y en en VehicleRepositoryImpl, haría el cambio. Comprobaría si el atributo reserve es false, si fuera así lo pasaría a true y viceversa.

Repository IMPL

En esta capa se separa la lógica de validación del mapeo y el acceso de datos. Aquí es donde vamos a realizar las validaciones. Por ejemplo si tenemos un update de un usuario, habrá que comprobar primero si el usuario existe, si esta sustituyendo una clave única que no se igual frente a las del resto de la base de datos y una vez estas comprobaciones se han realizado, se efectúa el cambio.

En caso de fallar en alguna de estas validaciones haríamos saltar una excepción, para el desarrollador muy útil a la hora de verlo en las herramientas del navegador y luego manejarlas en el front para que el usuario vea porque no se ha producido su acción.

```
public class VehicleRepositoryImpl implements VehicleRepository {

    private final VehicleJpaRepository vehicleJpaRepository;
    private final VehicleMapper vehicleMapper;

    @Override @Usage @Ivan
    @Transactional
    public Vehicle save(Vehicle vehicle) {
        //Llamamos al archivo validador creado para validar
        VehicleValidator.validateSave(vehicle);

        //Comprobamos en la base de datos si el id o la matricula ya existe.
        if (vehicle.getId() != null && vehicleJpaRepository.existsById(vehicle.getId())) {
            throw new IllegalArgumentException("Vehicle already exists");
        }

        if (vehicleJpaRepository.findByLicensePlate(vehicle.getLicensePlate()).isPresent()) {
            throw new IllegalArgumentException("License plate already exists");
        }
        //Asignamos al estado reserved false
        vehicle.setReserved(false);
        return vehicleMapper.toDomain(vehicleJpaRepository.save(vehicleMapper.toEntity(vehicle)));
    }
}
```

En el siguiente ejemplo podemos observar cómo funcionaría. Primero cuando son muchos campos los que hay que comprobar como en mi caso con el vehículo, lo pasamos por un archivo validador que creamos *proyecto muestra daw*. Podríamos validarlo todo aquí, pero llenaríamos de código y sería ilegible al final. Entonces solo validamos aquí lo que tenga que tener acceso a la base de datos, por eso llamamos al JpaRepository.

Este validador lo que haría sería validar que los campos introducidos no sean nulls o estén vacíos por ejemplo o en casos de fecha de creación, que no sea mayor a la de hoy en día, no puedes tener un vehículo del futuro. Que el kilometraje, precio, etc, no sea negativo.

Si esto ocurriera, saltaría la excepción y mostraría el mensaje indicando porque... Pues no puedes dejar un vehículo a la venta sin precio, o crear un vehículo sin marca.

Luego haría las comprobaciones a la bd, comprobaríamos por ejemplo que no existe ningún vehículo con esa id, aunque se autogenera, esta bien cubrir todas las posibilidades. También comprobaríamos que no puedes poner la misma matricula que otro, pues es única la matricula.

Por ultimo indicaríamos que ese vehículo su reserved es "false", pues un vehículo no puede salir a la venta estando ya reservado. Y ya realizamos la petición con los datos introducidos, se realiza un mapeo para introducirlo.

Ejemplo de validador.

```
public class VehicleValidator { 3 usages  @ Ivan
    //Creamos un archivo de validacion para no ensuciar de codigo excesivo en el IMPL

    public static void validateSave(Vehicle vehicle) { 1 usage  @ Ivan

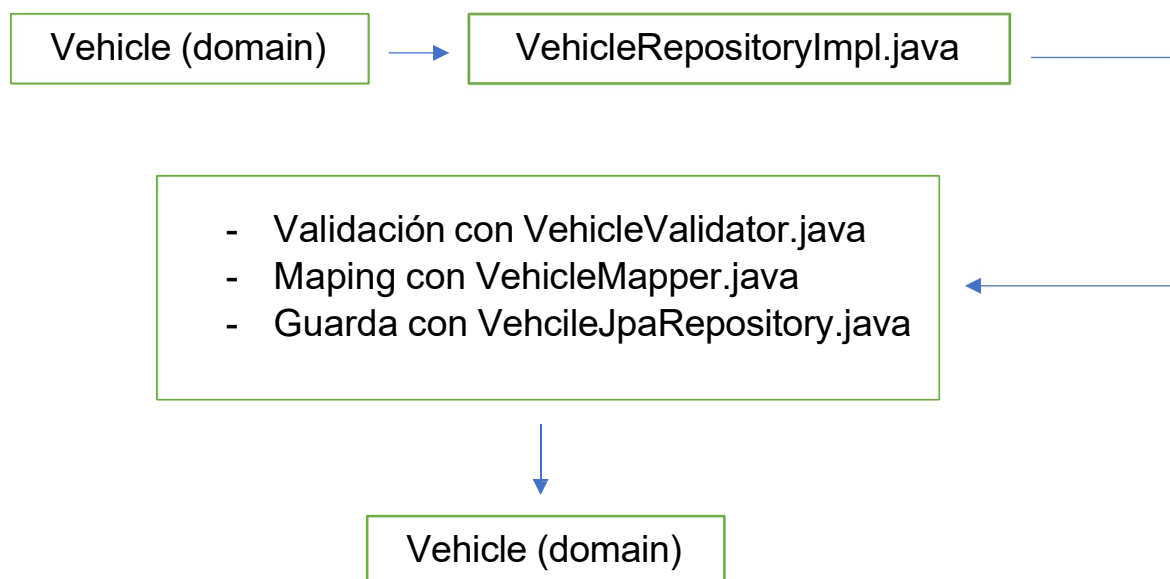
        if (vehicle == null) {
            throw new IllegalArgumentException("Vehicle cannot be null");
        }

        if (vehicle.getBrand() == null || vehicle.getBrand().isBlank()) {
            throw new IllegalArgumentException("Brand cannot be null or blank");
        }

        if (vehicle.getKm() == null || vehicle.getKm() < 0) {
            throw new IllegalArgumentException("Km cannot be null and not negative");
        }
    }
}
```

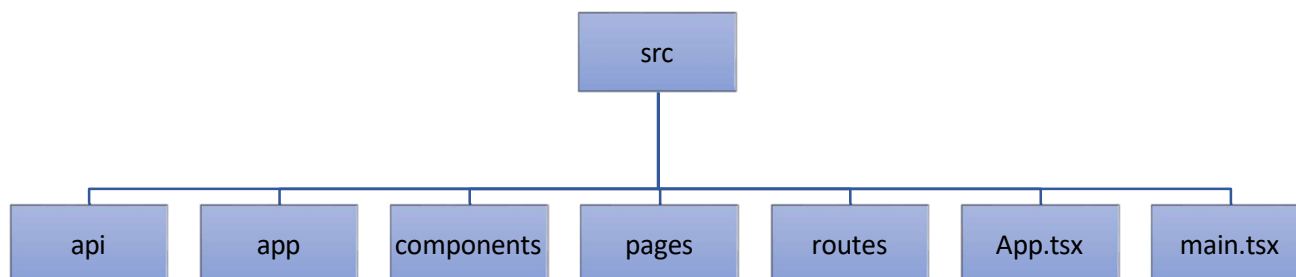
Para el resto de llamadas es muy similar. En update por ejemplo comprobaríamos si el id que nos están enviando existe y si es así, recuperaríamos de la base de datos *proyecto muestra daw* mediante ese id el objeto. De esta manera ya tendríamos disponible ese objeto para editar, volveríamos a introducir datos y volveríamos a realizar validaciones y se editaría.

Hay anotaciones que cabe mencionar como el @Transaccional, que se indica al principio de cada función. Asegura que cada operación que se haga, se realice correctamente y en caso de que no por cualquier cosa, se revierta.



3.4 Frontend.

Esquema



El frontend esta organizado de esta manera, modular, separando responsabilidades, por ejemplo como se puede observar rápido, pages por un lado y los componentes que pueden ser reutilizables en otras pages por otro lado.

API

Es donde se centralizan todas las llamadas HTTP al back. En este caso creamos dos carpetas user y vehicle. Dentro de cada una de ellas creamos un documento .ts en el que realizamos primero las interfaces de cada objeto, las interfaces para los updates si son objetos que modifican solo algunos datos y luego se ponen todas las llamadas al back.

De esta manera lo que se consigue es que no estemos en cada componente repitiendo interfaces o llamadas con lo que eso conlleva en cuanto a código. Lo escribimos una sola vez y cuando haya que realizar alguna llamada, se llama a la función pasándole el request o lo que sea necesario y ya está.

```
const API_URL = "http://localhost:8080/vehicles";

export const createVehicle = async (
  vehicle: CreateVehicleRequest,
): Promise<Vehicle> => {
  const response = await fetch(`${API_URL}/create`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify(vehicle),
  });
  if (!response.ok) {
    throw new Error("Failed to create vehicle");
  }
  return response.json();
};
```

En este ejemplo, crear vehículo, se puede observar el procedimiento para construir la llamada y luego en los componentes simplemente poner `createVehicle()` y simplificarlo.

Se indica su dirección, el tipo de método y el body, que son los datos del vehículo a crear.

```
export const getVehicleById = async (id: number): Promise<Vehicle> => {
  const response = await fetch(`${API_URL}/search_id/${id}`);
  if (!response.ok) {
    throw new Error("Failed to fetch vehicle");
  }
  return response.json();
};
```

Otro ejemplo, pero esta vez para obtener datos de un vehículo mediante su id. Como el método es GET no es necesario mencionarlo. Se pasa el id mediante la URL y obtenemos los datos en formato JSON que luego desengranamos para mostrarlo.

APP

En esta carpeta es donde realizamos el estado global y en los apartados anteriores ya explicados usando Redux y Hooks. Esta carpeta la subdividimos en dos más app/slices y app/store

- **Slices:** Aquí es donde configuramos los reducers y las versiones iniciales de cada objeto, initialState. Se crean los reducers necesarios, no necesariamente tiene que ser igual al número de llamadas a la API. Estos reducers al realizarles la llamada se encargaran de actualizar el estado. Si cuando realizamos la llamada a la API para crear un vehículo, realizamos también un dispatch *proyecto muestra daw* para llamar al reducer createVehicleSlice, estaremos añadiendo al estado que contiene la lista de vehículos actuales, ese vehículo nuevo, por lo que automáticamente nos aparecerá el vehículo en la lista. Si esto no sucediera, cada vez que realizáramos una llamada a la API tendríamos que estar recargando datos y haciendo llamadas a la API para volver a cargar la lista de vehículos por ejemplo en este caso. De esta manera no, se ha añadido el vehículo al estado, lo vemos y no hemos gastado mas recursos en volver a llamar a la API para traer de nuevo todos los vehículos y visualizarlos.

```
const initialState: VehicleState = {
  list: [],
  selectedVehicle: undefined,
  brands: [],
  models: [],
};

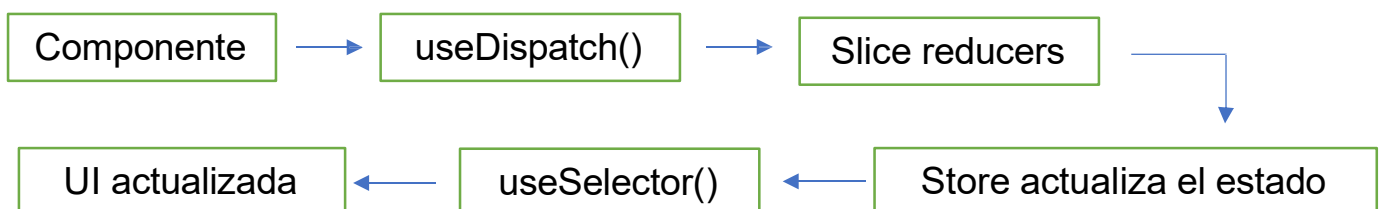
const vehiclesSlice = createSlice({
  name: "vehicles",
  initialState,
  reducers: {
    getAllVehiclesSlice(state, action: PayloadAction<Vehicle[]>) {
      state.list = action.payload;
    },
  },
});
```

En nuestro caso como son tres estado a actualizar, creamos tres slices, users, vehicles y session. En session es donde podemos mantener los datos del usuario actualmente logueado.

- **Store:** En la store es donde se centralizan todos los slices. Contiene el estado de todos los módulos. Cualquier componente puede acceder a la store y leer o modificar datos sin necesidad de crear interfaces o props en el componente.

También es donde se incluyen los hooks de React + Redux. Esto se usa en los componentes para comunicarse con la store.

- `useSelector` -> Lee el estado.
- `useDispatch` -> Dispara acciones. (Como se ha comentado antes con los reducers).
- `useEffect` -> Realizar llamadas a la API cuando uno de los estados o algún elemento cambia. De esta manera es como sabe cuando llamar a la API. Por ejemplo, ha cambiado el id de un vehículo, `useEffect` nota ese cambio y dispara una nueva llamada para realizar cualquier gestión que hayamos programado.



```
const session = useAppSelector((state) => state.session.user);
```

Un par de ejemplos. Gracias al estado de session, podemos traernos a cualquier componente la session, quien está conectado en ese momento. De esa manera no se realizan constantes llamadas y puedes llevar la información de un lugar a otro. En este caso la utilizo para comprobar en la UI si el usuario esta logueado, así le muestro cosa por pantalla o no...

El siguiente `useEffect` es perfecto para explicar, pues contiene al final lo que se mencionaba. Si `selectBrands` cambia (se añaden o eliminan marcas) automáticamente el `useEffect` se activa de nuevo, sin que nosotros tengamos que darle a ningún botón ni nada. De esta manera podemos conseguir que cuando el usuario selecciona una marca, automáticamente se realiza un `fetch` para traer los modelos de esa marca y que el usuario lo pueda visualizar.

En el `try` si todo es correcto se realiza la llamada y luego en el `catch` si ha surgido un error se envía un error, en forma de alerta para el usuario o un `console.log` en este caso para que el programador visualice que ha pasado.

```

useEffect(() => {
  const fetchModels = async () => {
    if (selectedBrands.length === 0) {
      setModels([]);
      setSelectedModels([]);
      return;
    }
    try {
      const dataModels = await getModelsByBrand(selectedBrands);
      setModels(dataModels);
      dispatch(getModelsByBrandsSlice(dataModels));
    } catch (error) {
      console.error("Failed to fetch models:", error);
    }
  };
  fetchModels();
}, [selectedBrands, dispatch]);

```

Components

Los componentes es una unidad reutilizable de la UI. Contiene las estructura en HTML que el usuario ve y la lógica. Hace que el proyecto sea modular y reutilizable. Pues por ejemplo puedo crear una carta de visualización de datos de vehículos que puedo usar en distintas páginas. De esta manera en cada pagina no vuelvo a realizar el mismo código para mostrar lo mismo.

Estos componentes contienen props, que es lo que necesita el componente para realizar la lógica que se ha programado y renderizar la UI para que todo funcione perfecto. Si mi carta necesita un id, porque realiza una llamada para buscar el vehículo, el prop en este caso es un id: Integer y cuando llamemos al componente habrá que pasarle un id.

Aquí es donde se usa todo lo que se ha mencionado antes, los hooks, las llamadas, los eventos para programar que cuando por ejemplo pulses un botón realice una acción.

```
const VehicleDetail = () => {
  const navigate = useNavigate();
  const { id } = useParams();
  const session = useAppSelector((state) => state.session);
  const vehicle = useAppSelector((state) =>
    state.vehicles.list.find((v) => v.id === Number(id)),
  );

  const handleDelete = () => {
    if (window.confirm("Are you sure you want to delete this vehicle?")) {
      deleteVehicle(Number(id));
      alert("Vehicle deleted successfully");
      console.log("Vehicle deleted");
      navigate("/");
    }
  };

  const handleReserve = () => {
    reserveVehicle(Number(id));
  };
};
```

Este ejemplo no contiene props, pues es un componente más de visualización como su nombre indica. VehicleDetail, un componente donde visualizamos los detalles del vehículo seleccionado.

A este componente accedemos mediante una ruta, en esa ruta hemos pasado el id que lo capturamos con el useParams().

También se puede ver la importancia de los estados de Redux. Como veníamos de una lista de vehículos, todos esos vehículos están cargados en el estado, pues pasamos el id al estado de la lista y obtenemos el vehículo, sin necesidad de gastar recursos en una llamada a la API para obtener la información del vehículo seleccionado.

Se puede también varios eventos para que cuando quieras realizar la acción de borrar o reservar en este caso el vehículos. Se puede ver que en ese caso ya si que se realiza la llamada a la API y se realiza la acción. En caso de si se borrara se puede observar que mediante el useNavigate() volvemos a la URL que se haya programado. Pues si borramos el vehículo deja de existir asique volvemos al home.

Pages

En esta carpeta es donde están las vistas completas de la aplicación. Pues si por ejemplo nuestra page Home queremos que visualice el sistema de filtros y visualización de la lista de vehículos, esta lista llamamos a esos dos componentes. De esta manera todo queda mas organizado sin mezclar funciones.

```
function Home() {  
  const [filters, setFilters] = useState<VehicleFilter>({});  
  
  return (  
    <div className="home">  
      <div className="filter_container">  
        <FilterVehicle onFilter={setFilters} />  
      </div>  
      <div className="list_container">  
        <ListAllVehicles filters={filters} />  
      </div>  
    </div>  
  );  
}  
  
export default Home;
```

Como se puede observar, sucede lo que se comentaba antes, se llama a los componentes y se le pasan sus props, en este caso FilterVehicles pasa los filtros para que luego ListAllVehicles los use, renderice y haga las llamadas necesarias par mostrar los vehículos que están dentro de los parámetros de filtrado.

Routes

Contiene el archivo AppRoutes.tsx. Es el archivo que define las rutas de la aplicación con React Router. Es quien permite la navegación entre páginas. Posibilidad de proteger las rutas mediante el login. En este caso no lo utilizo porque mi aplicación es apta para usuarios que tengan cuenta y que no.

```
export default function AppRoutes() {
  return (
    <Router>
      <Navbar />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/vehicle/:id" element={<VehicleDetail />} />
        <Route path="/mylist" element={<FavouritesPage />} />
        <Route path="/account" element={<AccountPage />} />
        <Route path="/createVehicle" element={<CreateVehiclePage />} />
        <Route path="/createVehicle/:id" element={<CreateVehiclePage />} />
      </Routes>
    </Router>
  );
}
```

Se puede observar que cada ruta lleva a una page o en alguno en especial a un componente de visualización. De manera que si en la URL pones la dirección te lleva a la pagina. Una futura mejora es proteger las rutas mediante roles. Mediante la instalación e implantación del sistema de keycloak, se realizaría.

Main.tsx

Es el punto de entrada en la aplicación. Es quien renderiza el componente App en el DOM con todas sus rutas.

```
ReactDOM.createRoot(document.getElementById("root") as HTMLElement).render(  
  <React.StrictMode>  
    <Provider store={store}>  
      <App />  
    </Provider>  
  </React.StrictMode>,  
);
```

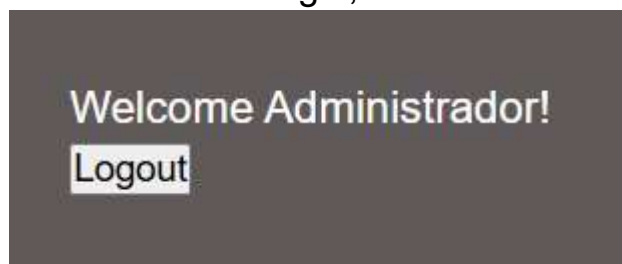
3.5 Visualización web.

Navbar



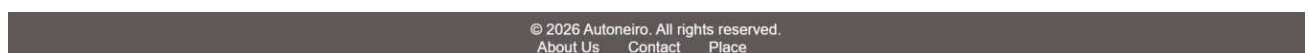
De izquierda a derecha se ve como tenemos el nombre de la empresa o logo. Es un link que lleva a la page Home. My List y Account son otros dos enlaces. El primero te lleva a la lista personal de coches favoritos y el segundo a tu cuenta, para que te crees una cuenta, la edites o la borres.

Lo último es el Login, donde el usuario mete sus credenciales y se loguea.



En caso de estar ok las credenciales pasaría a este estado.

Footer



Es el pie de pagina que tanto el navbar como el, salen en todas las paginas de la aplicación. Donde se ven que están PROYECTO MUESTRA DAW los derechos registrados y tres enlaces en los que te lleva a conocer sobre la empresa, el contacto y el lugar de la sede.

Vehicle Detail

Autoneiro

My ListAccount

Welcome Ivan Ameneiro!
Logout

Toyota Land Cruiser

Price: 22000 €
Power: 173 cv
Engine capacity: 4200 cc
Km: 160000 km
Fuel: DIESEL
Color: Green
Doors: 5
Transmission: MANUAL
Function: WORK
Description: Vehículo robusto para cualquier terreno

Remove from favs

© 2026 Autoneiro. All rights reserved.
About UsContactPlace

Autoneiro

My ListAccount

Welcome Administrador!
Logout

Toyota Land Cruiser

Price: 22000 €
Power: 173 cv
Engine capacity: 4200 cc
Km: 160000 km
Fuel: DIESEL
Color: Green
Doors: 5
Transmission: MANUAL
Function: WORK
Description: Vehículo robusto para cualquier terreno

Add to favs

Edit

Delete

Reserved

© 2026 Autoneiro. All rights reserved.
About UsContactPlace

My List

Autoneiro

My ListAccount

Welcome Administrador!
Logout

My Favourites

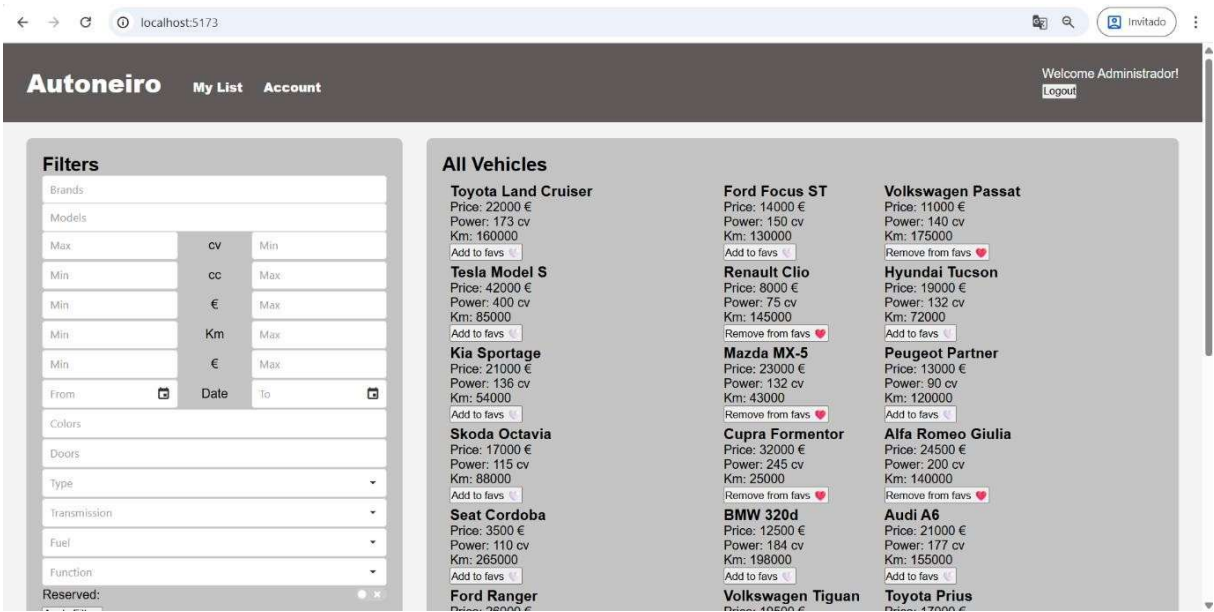
Mazda MX-5
Price: 23000 €
Power: 132 cv
Km: 43000
Remove from favs

Volkswagen Passat
Price: 11000 €
Power: 140 cv
Km: 175000
Remove from favs

Honda CR-V Hybrid
Price: 32000 €
Power: 184 cv
Km: 40000
Remove from favs

Alfa Romeo Giulia
Price: 24500 €
Power: 200 cv
Km: 140000
Remove from favs

General



Create vehicle

Autoneiro My List Account Welcome Administrator! Logout

Create

Brand:

License Plate:

Power:

Km:

Doors:

Transmission:

Fuel:

Description:

Model:

Price:

Engine Capacity:

Data Created:

Color:

Type:

Function:

Create

© 2026 Autoneiro. All rights reserved.

Edit vehicle

Autoneiro

My ListAccount

Welcome Administrator!

Logout

Edit Toyota Land Cruiser

Price:22000

Km:160000

Doors:5

Type:4x4

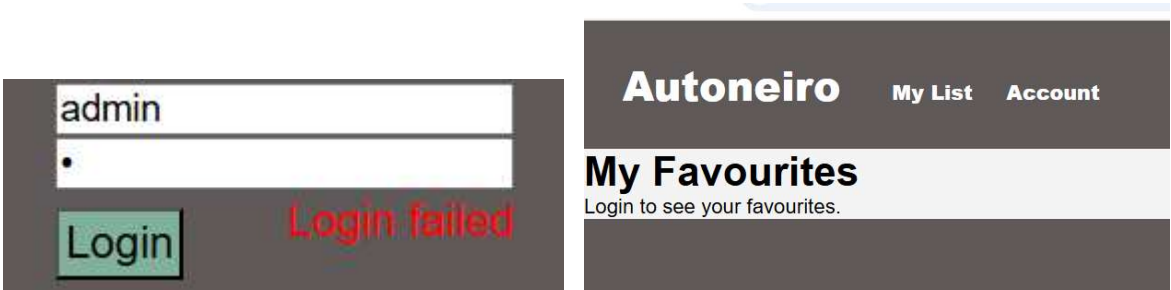
Description:Vehículo robusto para cualquier terreno

Update

© 2026 Autoneiro. All rights reserved.

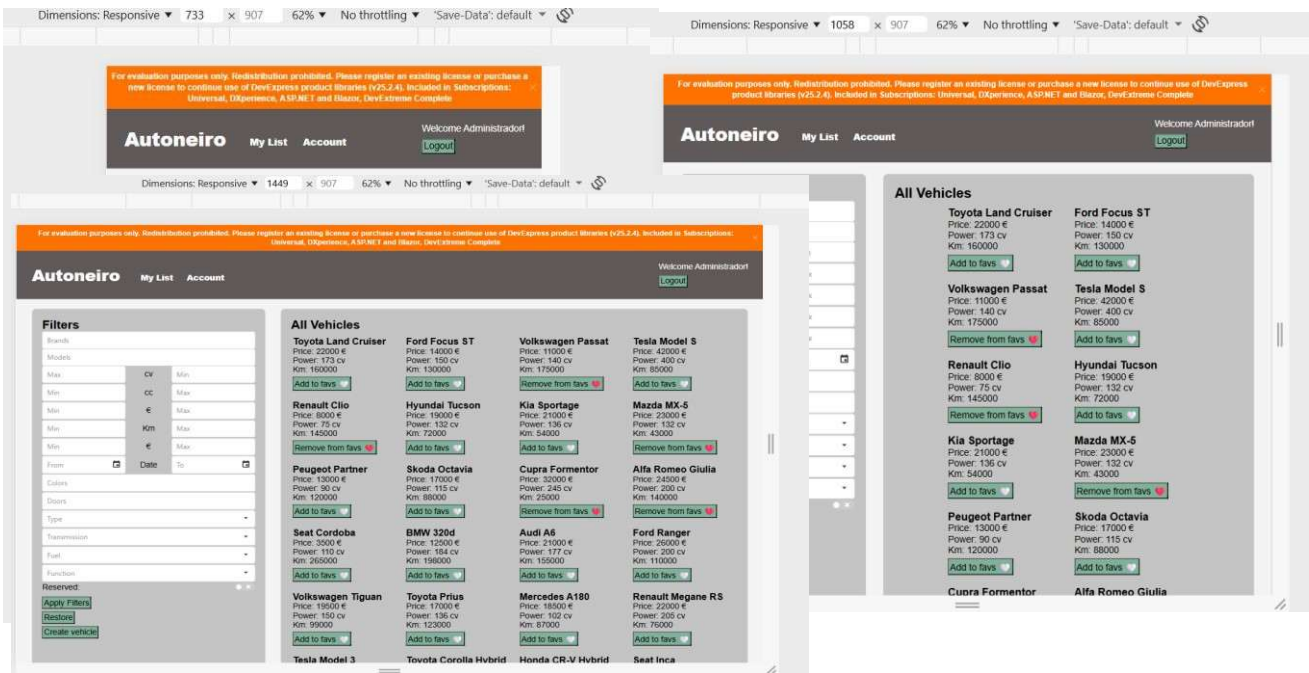
About UsContactPlace

Error



También existe mensajes de carga pero que son difíciles de capturar, en el momento de la exposición se detallaran.

Responsive



4. Fases en la realización

4.1 Planificación inicial.

Se planeo una aplicación que destacara un poco por el sistema de filtrado y una fácil utilización de la aplicación. Hay muchas webs de este tipo de negocio, pero el planteamiento inicial era ese, destacar añadiendo mas filtros sin que el usuario tuviera que ser asesorado por un especialista.

Se inicio primero realizando un esquema relacional y una lista de endpoints que pensaba obligatorios para el correcto funcionamiento. A medida que ha ido creciendo la app, se ha ido añadiendo alguno mas pero los principales están.

4.2 Desarrollo del backend

Lo primero de todo fue instalar los programas necesarios y crear los contenedores para la base de datos.

Se definieron las entidades y una vez se crearon en este caso las dos entidades se definió un slice primero entero.

1º. Ports in y out

2º. Service

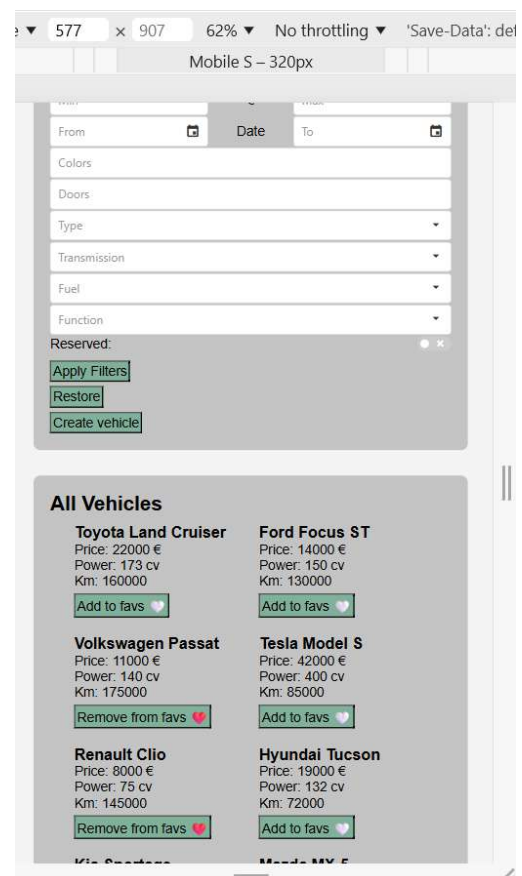
3º. Infrastructure / entity

4º. Mapper

5º. Repository

6º. Repository IMPL

7º. Controller



Una vez terminados los dos slices usando este orden, se comenzó a realizar el flyway generando las tablas.

4.3 Desarrollo del frontend.

Se creo el contenedor y se empezó a realizar la configuración del programa con Vite. Se instalan dependencias necesarias.

```
"@reduxjs/toolkit": "^2.11.2",  
"axios": "^1.13.5",  
"devextreme": "^25.2.4",  
"devextreme-react": "^25.2.4",  
"react": "^19.2.0",  
"react-dom": "^19.2.0",  
"react-redux": "^9.2.0",  
"react-router-dom": "^7.13.0",  
"react-switch": "^7.1.0"
```

Una vez con todo preparado se procede por el siguiente orden a realizarlo.

- 1º. Creación de la store y slices
- 2º. Creación de hooks
- 3º. Creación de componentes principales como ListAllVehicles o FilterVehicles
- 4º. Creación de pages e implementación de los componentes.
- 5º. Creación visual de navbar y footer
- 6º. Creación visual del login e implementación en navbar.
- 7º. Diseño de filtros y campos de creación/edición de vehículos

4.4 Pruebas

Se comienzan las pruebas realizando el login con un usuario normal y el user administrador. Se comprueba que la UI es diferente teniendo el rol de administrador o el de usuario.

Se comprueban los filtros, se realizan todos los filtros uno a uno y también buscando uno específicamente y comprobando que funciona correctamente.

Se comprueban que los errores se muestran cuando te equivocas en el login, que se muestran los mensajes de alertas a la hora de crear, modificar o borrar vehículos.

Errores detectados

- No podía manejar las excepciones.
- Cuando se crea el vehículo se crea con la fecha del día actual y no con la que introduce el administrador.
- Fallo visual en la distribución de vehículos en la lista.
- Fallo en la visualización de los campos de creación, ocupaban un tercio de la pantalla y no el total designado.

5. Ampliaciones o mejoras

El sistema creado actualmente cumple con todos los requerimientos básicos que se pensaron para la aplicación. Durante la creación de esta se han ido encontrando alguna mejora que se ha podido implementar y otras para estudiar e implementar.

Keycloak

Es un sistema de gestión de autenticación y autorización de usuarios. El sistema actualmente funciona con una autenticación muy básica, pues se comprueba con la base de datos si el user que se esta logueando existe. Este sistema no es el mas optimo, pues estas pasando datos críticos que podrían capturar. También mejoraría la gestión de roles, pues ahora cualquier usuario usando la url, podría crear un vehículo lo único que hacemos es mostrar o no mostrar visualmente dependiendo del rol que tengas.

Este nuevo sistema permitiría:

- Gestión segura de usuarios.
- Control de roles.
- Autenticación mediante token.
- Mayor seguridad en los endpoints y usar preAuthorize para roles.

Gestión de imágenes de vehículos

Actualmente los vehículos que he implantado no tienen imágenes. En la aplicación cuando estuviera mas avanzada una mejora seria añadir una biblioteca de imágenes. Cada vez que entraras a un vehículo en concreto pudieras mostrar el estado del vehículo y como eso.

- Implementar subida de archivos desde el front
- Mostrar imágenes

Obtención de marcas de vehículos desde internet

Ahora mismo para la creación del vehículo se escribe la marca, puesto que no es datos que se debiera hardcodear una lista de marcas, se podría conseguir de internet y realizar un select, de la misma manera que hemos hecho en el filtrado. Se obtendrían dinámicamente y aunque no tuviéramos ese vehículo podríamos crearlo.

- Reduce errores humanos. Pues si escribes mal la marca dará fallo después del programa. Usando select esto no ocurrirá.
- Mantenemos datos actualizados.
- Se automatizan los formularios.

Sistema de logs

Se podría implementar un sistema en el que cada vez que se vendiera un vehículo se creara un log. De esta manera tendríamos un registro de los vehículos que se han vendido. Usuario, precio, fecha de venta y vehículo. Este sistema ayudaría mucho a la parte financiera del negocio, mas control de gasto y beneficio.

- Trazabilidad
- Auditoría interna
- Análisis

Exportación a Excel

La incorporación de esta funcionalidad nos permitiría generar informes y descargarlos, conseguiríamos un mayor control y facilitaría la gestión para otros departamentos de la empresa.

- Listado de vehículos en stock
- Historial de ventas
- Beneficios totales

Implantación de traducciones

Mediante un botón cambiar toda la UI de idioma. Lo realice durante las practicas y es una de las mejores funciones que pude tener un programa. Puede ser una buena mejora para esta app.

6. Conclusiones

Este proyecto me ha permitido repasar mis conocimientos sobre esta materia, pues solo había realizados dos proyectos completos parecidos a este. Salgo con las ideas más claras y entendiendo aun mas alguna tecnología.

A lo largo del proyecto se ha trabajado en diseño de arquitectura, implantación de lógica de negocio y en construir una interfaz web.

6.1 Lecciones aprendidas.

He comprobado que una correcta planificación al principio de realizar el programa, ayuda mucho durante la ejecución de esta. Seguir los planes y la directrices establecidas.

Aumentar el conocimiento sobre Redux.

La gestión de errores y validaciones han sido otra de las cosas que he aprendido en mayor medida, pues de esta manera se a conseguido que la aplicación sea muy optima y que no de lugar a errores o bugs y si se realiza algo mal que se muestre porque y como solucionarlo.

7. Bibliografía

- <https://react.dev/>
- <https://redux-toolkit.js.org/introduction/getting-started>
- https://axios-http.com/docs/api_intro
- https://js.devexpress.com/React/Documentation/Guide/React_Components/DevExtreme_React_Components/
- <https://app.diagrams.net/>
- <https://ed.team/blog/que-es-react-y-por-que-domina-el-desarrollo-frontend>
- Proyectos de las practicas
- YouTube